

SOP 90 — Self-Annealing (auto-cicatrização)

Camada A.N.I.: Architecture (processo) · Versão: 1.0 (Protocol 0, 2026-09-09)

Invariante 6: erro → ler stack → patch → testar → registrar o aprendizado no SOP.

Objetivo

Definir o ciclo obrigatório de recuperação de falhas e **registro de aprendizado** do sistema: nenhuma tool quebra em silêncio, nenhum patch entra sem teste contra fixtures e nenhuma lição fica sem dono (o SOP correspondente é atualizado **antes** do código, Golden Rule).

Quando roda

- Toda vez que uma tool falhar (exceção, saída inválida, integração fora do ar, validação de schema rejeitada).
- Toda vez que uma lógica mudar (mesmo sem erro): SOP primeiro, código depois.
- Revisão contínua: alertas de erro vão para o webhook do editor (`EDITOR_WEBHOOK_URL`).

Input

- Stack trace / log da tool que falhou (sempre com o JSON de entrada que causou a falha).
- Estado atual do SOP da tool + fixtures relacionados.
- Histórico: `progress.md` (erros e testes) e `findings.md` (lacunas conhecidas).

Output

- Patch aplicado e **testado**; SOP da tool atualizado com o aprendizado (seção de edge cases/erros conhecidos).
- Registros em `progress.md` (data, tool, erro, patch, teste) e no Maintenance Log de `gemini.md` (fase T).
- Alerta ao editor quando a falha impedir entrega do dia.

Passos determinísticos (ciclo)

1. **Detectar:** tool retornou erro/status inesperado → capturar stack + entrada + saída parcial; alertar editor via webhook se for falha de entrega (payload do dia).
2. **Ler o stack:** identificar a camada (rede/schema/lógica/cloud). Não “consertar” às cegas: reproduzir com o mesmo input.
3. **Classificar:**
 - Falha de integração (API fora, quota, credencial) → registrar em `progress.md` / `findings.md` ; tool degrada conforme o SOP dela (mock/retry) e segue — a integração não bloqueia as outras.
 - Falha de lógica/schema → bug do nosso código: vai para o passo 4.
 - Falha de dado (input inválido vindo de fonte) → tratar como edge case no SOP da tool (ex.: descartar + logar), sem mudar regra de negócio.

4. **Patch:** mudança **mínima** na tool; **antes** de qualquer código, atualizar o SOP correspondente (Golden Rule). Nunca mudar regra de negócio do blueprint/gemini.md por conta própria — dúvida → PARAR e perguntar ao humano.
5. **Testar:** rodar a tool contra os fixtures (`fixtures/`) + o caso que quebrou. Teste passou? Só então o patch vale. Teste falhou → voltar ao passo 4.
6. **Registrar o aprendizado:** adicionar o caso à seção de edge cases do SOP; anotar em `progress.md` (formato abaixo); atualizar Maintenance Log quando em produção (fase T).
7. **Rollback se necessário:** se o patch não estabilizar em 2 tentativas, reverter para a última versão estável da tool (versão testada anterior) e subir o problema para o humano com o diagnóstico — sem insistir no mesmo patch (regra de parada).

Edge cases

- Falha repetida da mesma integração 3 ciclos seguidos → **não** continuar tentando em loop: marcar integração como degradada/offline em `findings.md` e notificar humano.
- Fixture desatualizado (mudou o schema na CONSTITUTION) → atualizar fixture junto com SOP e código no mesmo patch (schema é contrato).
- Duas tools falham na mesma esteira → tratar na ordem da Navigation (upstream primeiro: ingest → classificação → build → publish).
- Erro só em produção (funciona nos fixtures) → reproduzir com o input real salvo em `.tmp/` e criar um caso de teste novo a partir dele (regressão).
- Patch que altera comportamento visível ao torcedor (tom, selo, seção) → além do SOP, passar pela fila do editor (nunca auto-deploy de mudança editorial).

Rate limits / limites conhecidos

- Retries máximos por evento: 3 com backoff (5s/15s/30s) para integrações; polling de jogo segue o SOP 31 (30s).
- Alerta de erro: 1 mensagem por incidente (evitar spam no canal do editor).

Testes

- Rotina de auto-teste: toda tool roda em modo `--fixture` antes de qualquer execução real (parte do Definition of Done).
 - Checklist pós-patch: (1) SOP atualizado? (2) fixture passou? (3) caso real que quebrou passou? (4) `progress.md` registrado? (5) editor notificado se entrega afetada?
 - O sistema “anneals” quando, após uma falha, a próxima execução do mesmo cenário passa **sem** mudança manual além do patch registrado.
-

Anexo — formato de registro de erro (usar em progress.md)

```
- YYYY-MM-DD HH:MM -03 | erro | tool=<nome> | fase=<fase> |  
  sintoma: <o que aconteceu>  
  causa provável: <hipótese após ler o stack>  
  patch: <mudança mínima aplicada>  
  teste: <fixture/caso usado + resultado>  
  sop_atualizado: <architecture/arquivo.md>
```